

# Tutorial da Aula 01

Primeira aplicação web no NetBeans sem Maven

**NETBEANS**

**ANT**

**TOMCAT**

**Tecnologias: HTML, JSP, Jakarta Servlet e Apache Tomcat**

**Requisito prévio: orientação a objetos em Java**

Material didático com telas ilustrativas e marcações de onde clicar no NetBeans.

# 1. Organização da aula

Este tutorial conduz a turma desde a configuração do servidor até a execução de uma aplicação que recebe nome e idade, cria um objeto Java e apresenta o resultado em uma página JSP.

Duração sugerida: 3 horas

## Objetivos de aprendizagem

- Compreender o modelo cliente-servidor e o fluxo requisição-resposta.
- Diferenciar aplicações Java desktop e aplicações Java Web.
- Configurar o Apache Tomcat no NetBeans.
- Criar um projeto Web construído pelo Ant, sem Maven.
- Receber parâmetros com um Servlet e encaminhar dados para uma JSP.
- Relacionar encapsulamento, objetos e métodos ao desenvolvimento web.

## Roteiro de tempo

| Etapa                         | Tempo  |
|-------------------------------|--------|
| Fundamentos de web e HTTP     | 45 min |
| Configuração do Tomcat        | 20 min |
| Criação do projeto Ant        | 20 min |
| Implementação guiada          | 70 min |
| Testes, validação e exercício | 25 min |

## 2. Fundamentos de aplicações web

Em uma aplicação Java convencional, o programa é iniciado e executado diretamente no computador do usuário. Em uma aplicação web, o código Java é executado no servidor. O navegador envia requisições e apresenta as respostas recebidas.

**Fluxo básico: Navegador → requisição HTTP → Tomcat → Servlet → resposta HTTP → navegador**

### Papéis principais

| Elemento         | Responsabilidade                                               |
|------------------|----------------------------------------------------------------|
| Navegador        | Atua como cliente, envia solicitações e apresenta HTML.        |
| Apache Tomcat    | Recebe requisições e executa Servlets e páginas JSP.           |
| Servlet          | Classe Java que controla e processa a requisição.              |
| JSP              | Página responsável por apresentar dados em HTML.               |
| Classe de modelo | Representa dados e regras do domínio com orientação a objetos. |

Exemplos de aplicações web incluem sistemas acadêmicos, lojas virtuais, sistemas bancários, plataformas de ensino e sistemas administrativos.

### 3. HTTP, GET e POST

HTTP é o protocolo utilizado na comunicação entre o navegador e o servidor. Uma requisição informa um endereço, um método, cabeçalhos e, dependendo do método, dados no corpo.

#### Exemplo de endereço

```
http://localhost:8080/PrimeiraAplicacaoWeb/cadastro
```

| Parte                | Significado                            |
|----------------------|----------------------------------------|
| http                 | Protocolo de comunicação.              |
| localhost            | O servidor está no próprio computador. |
| 8080                 | Porta utilizada pelo Tomcat.           |
| PrimeiraAplicacaoWeb | Contexto da aplicação.                 |
| cadastro             | Caminho associado ao Servlet.          |

#### Método GET

É usado principalmente para consultar recursos. Parâmetros podem aparecer no endereço. Não deve ser utilizado para modificar dados importantes nem para transmitir informações sensíveis.

#### Método POST

É usado para enviar dados ao servidor, como cadastros e formulários. Os dados seguem no corpo da requisição. O POST, por si só, não criptografa os dados; a proteção durante a transmissão depende de HTTPS.

## 4. Preparação do ambiente

Utilize JDK 17 ou superior, Apache NetBeans, Apache Tomcat 11 e um navegador. O Tomcat 11 utiliza pacotes `jakarta.servlet`. Códigos antigos com `javax.servlet` não devem ser copiados para este projeto.

### Adicionar o servidor ao NetBeans

Abra Window → Services. Em Servers, clique com o botão direito e escolha Add Server....



Figura 1 - Abrindo o assistente para adicionar um servidor.

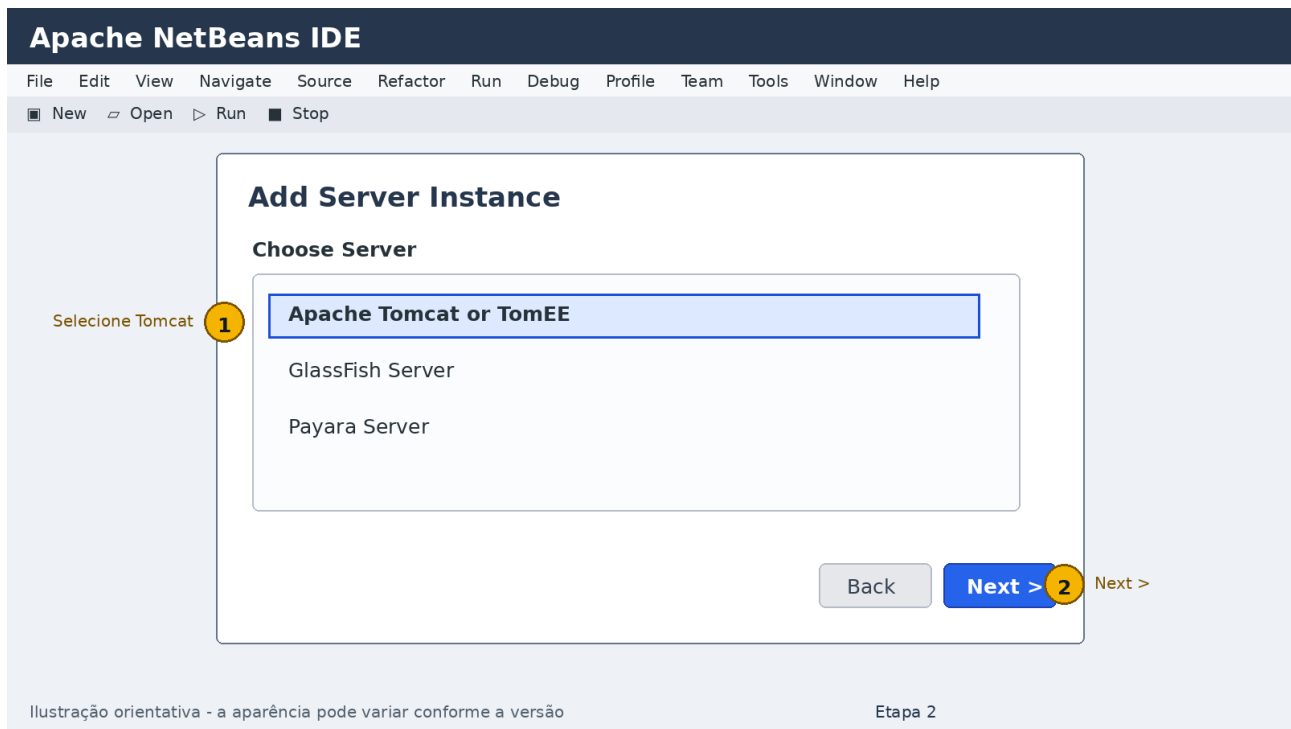


Figura 2 - Selecionando Apache Tomcat or TomEE.

Selecione Apache Tomcat or TomEE e avance em Next.

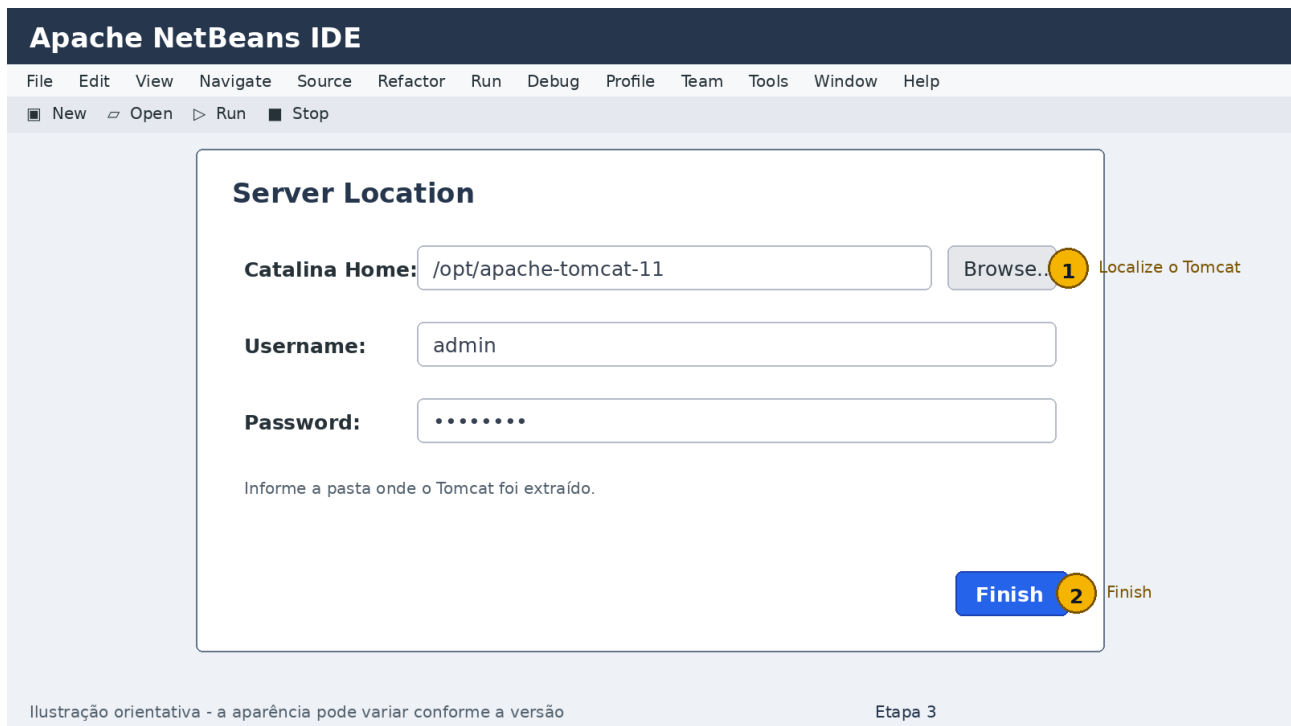


Figura 3 - Informando a pasta de instalação do Tomcat.

Em Catalina Home, indique a pasta em que o Tomcat foi extraído. Informe usuário e senha apenas se o assistente solicitar. Finalize a configuração.

**Possível problema: se o NetBeans não aceitar a pasta, confira se ela contém as subpastas bin, conf, lib e webapps.**

## 5. Criar o projeto Web com Ant

Acesse File → New Project. Escolha a categoria Java Web e o projeto Web Application.

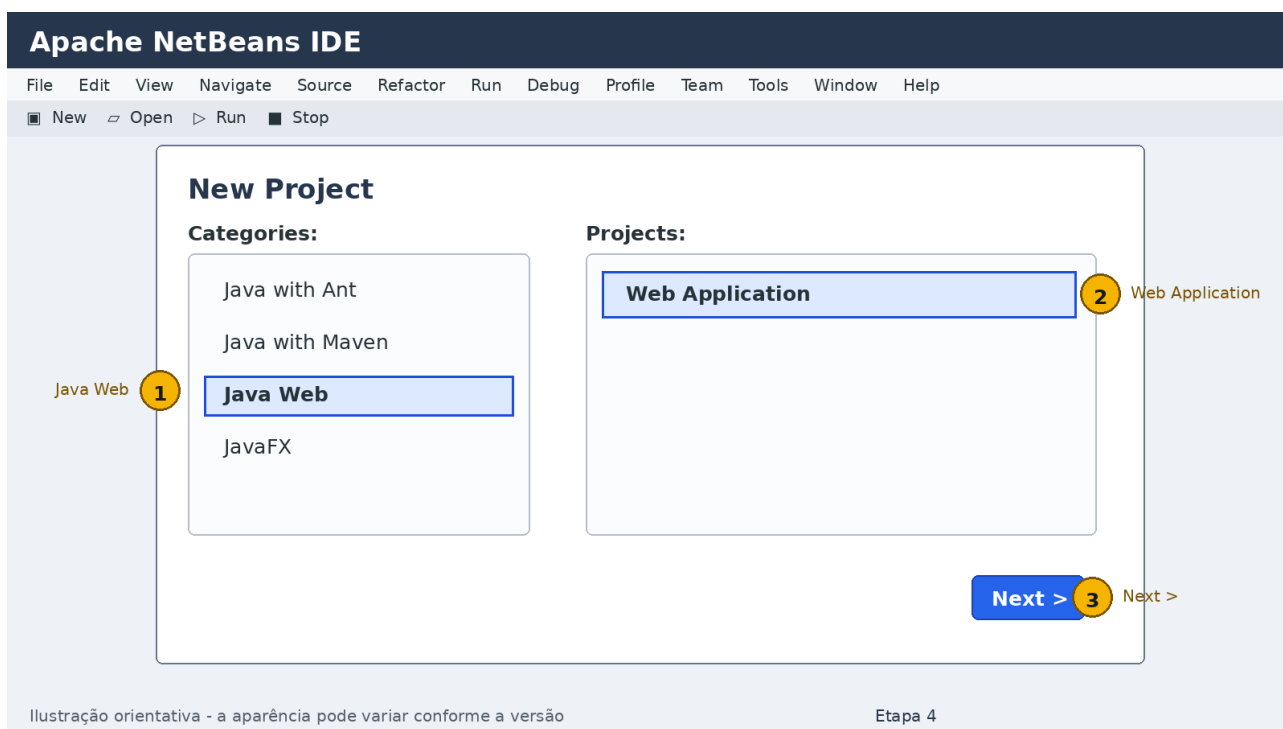


Figura 4 - Criando uma Web Application tradicional, sem Maven.



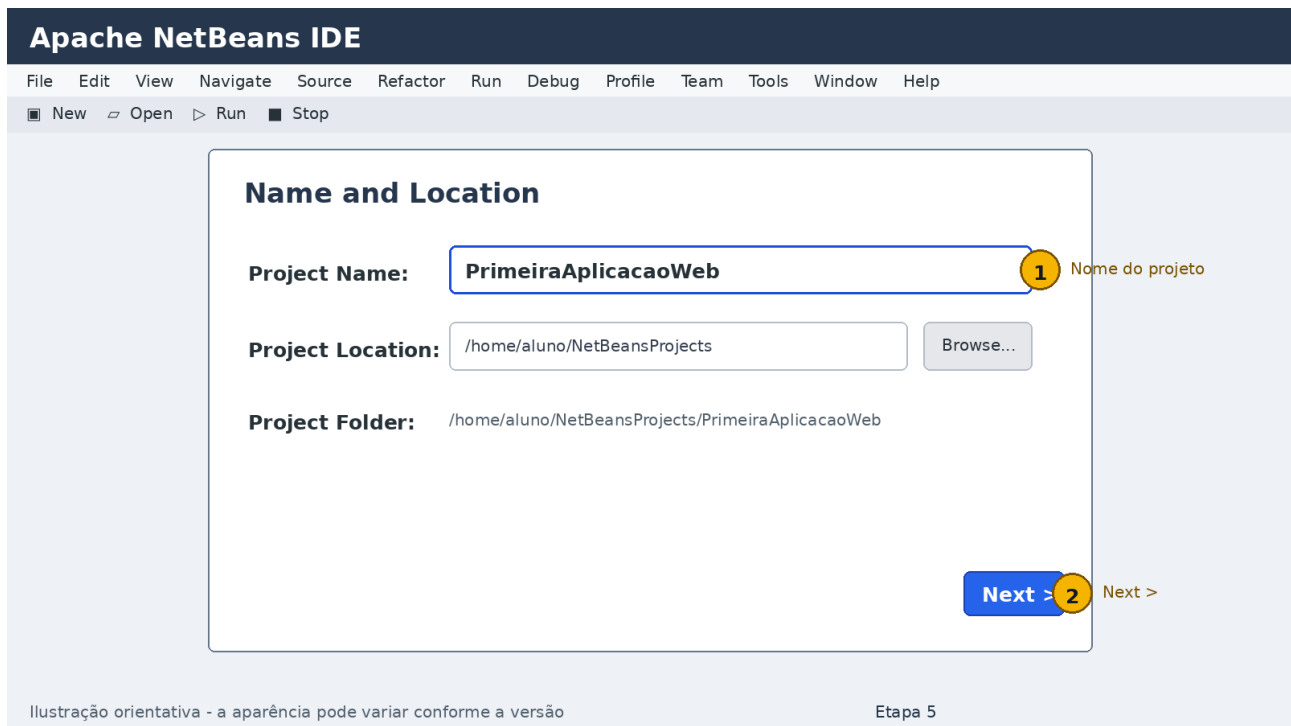


Figura 5 - Definindo o nome PrimeiraAplicacaoWeb.

Informe PrimeiraAplicacaoWeb. A criação tradicional desta categoria utiliza Ant e produzirá o arquivo build.xml.

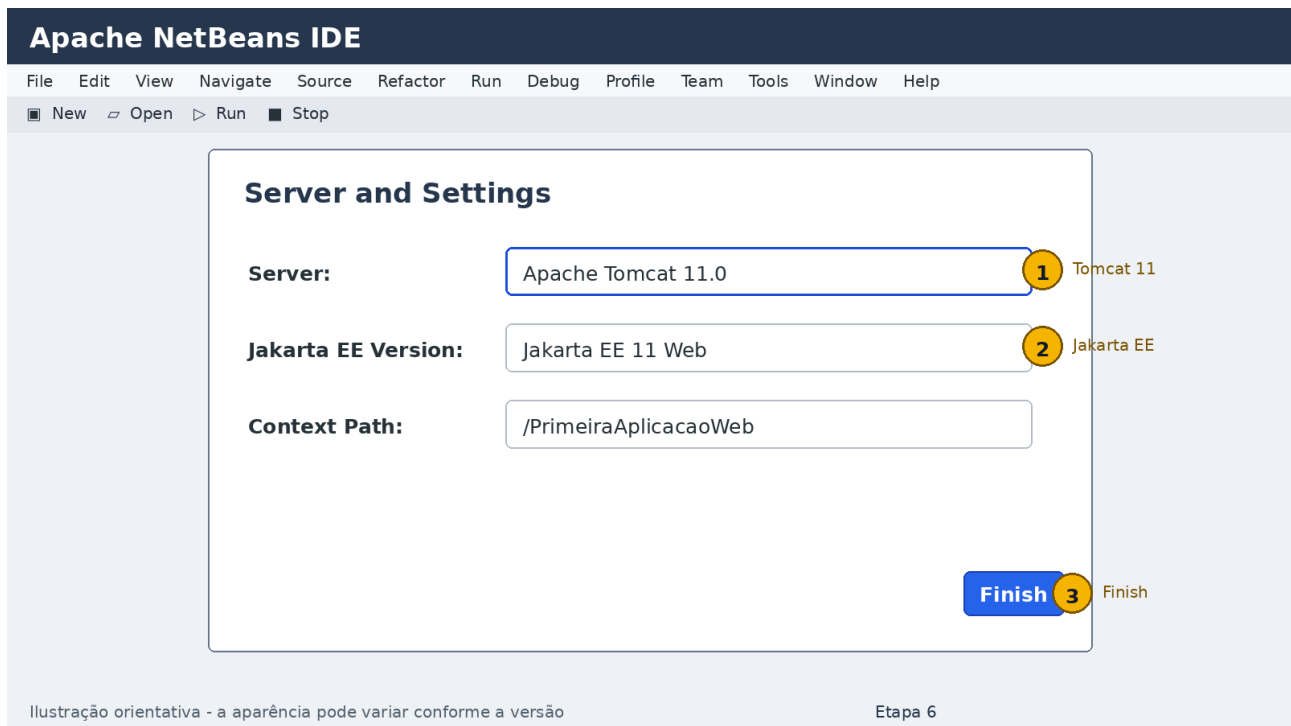


Figura 6 - Selecionando o Tomcat, a versão Jakarta EE e o contexto.

Selecione o Tomcat 11, escolha a versão Jakarta EE Web compatível oferecida pelo NetBeans e mantenha o contexto `/PrimeiraAplicacaoWeb`.

**Importante: não escolha a categoria “Java with Maven”. O projeto desta aula é gerenciado pelo Ant e pelas bibliotecas fornecidas pelo servidor.**

## 6. Estrutura do projeto



Figura 7 - Principais áreas do projeto no NetBeans.

| Elemento            | Uso                                                    |
|---------------------|--------------------------------------------------------|
| Source Packages     | Classes Java, modelos e Servlets.                      |
| Web Pages           | HTML, JSP, CSS, JavaScript e imagens.                  |
| Libraries           | Bibliotecas adicionadas pelo NetBeans e pelo servidor. |
| Configuration Files | Arquivos de configuração da aplicação.                 |
| build.xml           | Tarefas Ant para compilar, empacotar e executar.       |
| dist                | Pasta criada na construção; recebe o arquivo WAR.      |

O arquivo WAR é o pacote distribuível da aplicação web. O NetBeans o produz ao executar Clean and Build.

## 7. Criar a classe de modelo

Em Source Packages, clique com o botão direito e escolha New → Java Package.

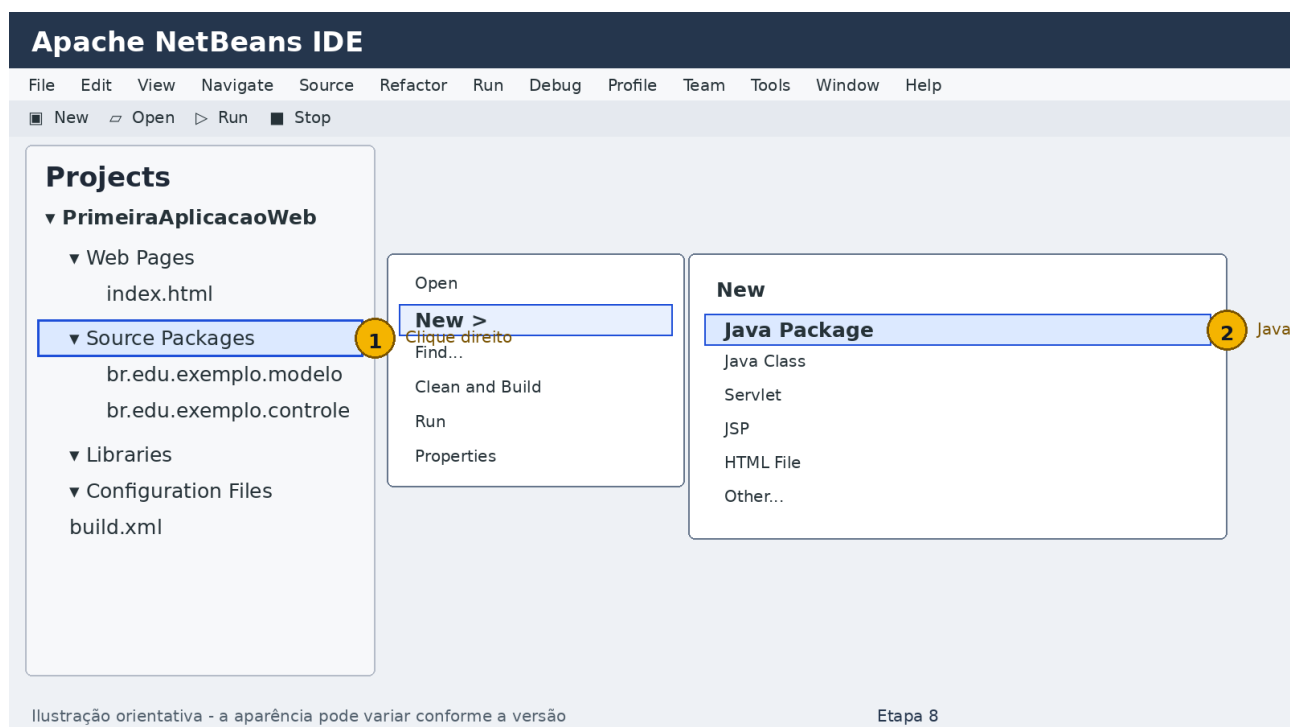


Figura 8 - Criando um pacote Java.

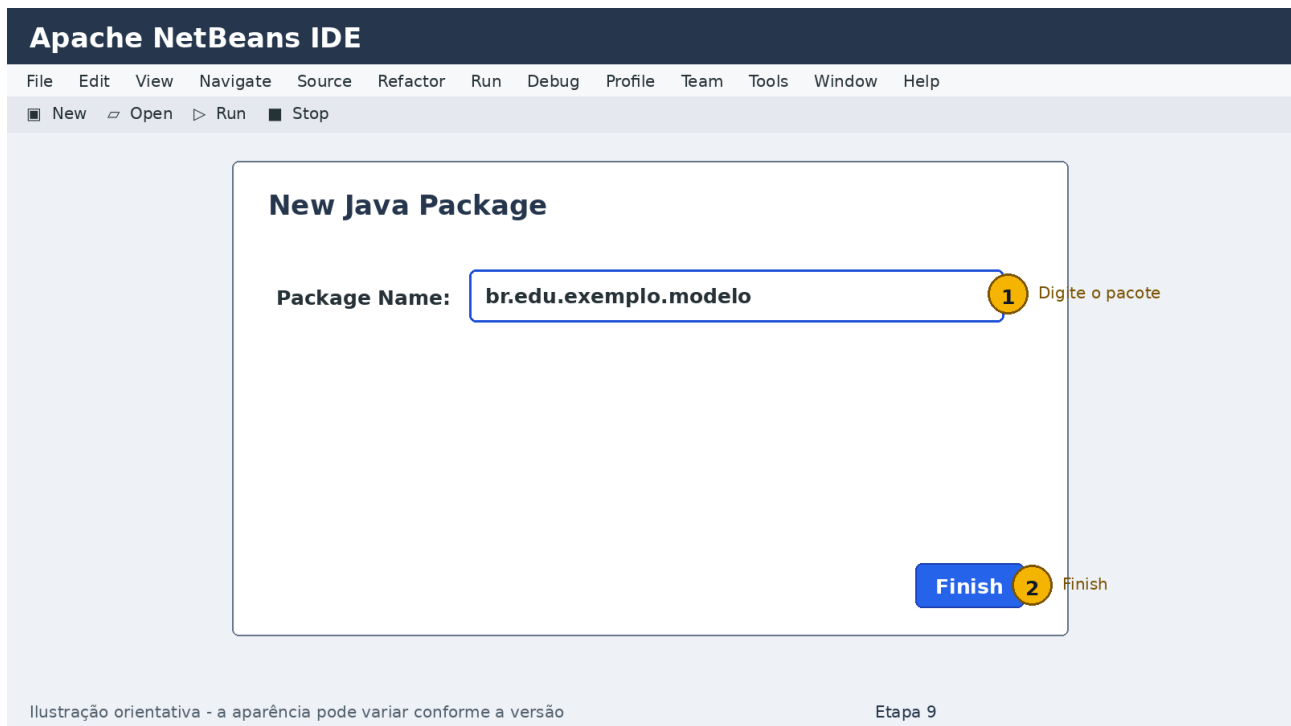


Figura 9 - Nomeando o pacote do modelo.

Crie o pacote `br.edu.exemplo.modelo`. Depois, clique com o botão direito nesse pacote e selecione `New → Java Class`.

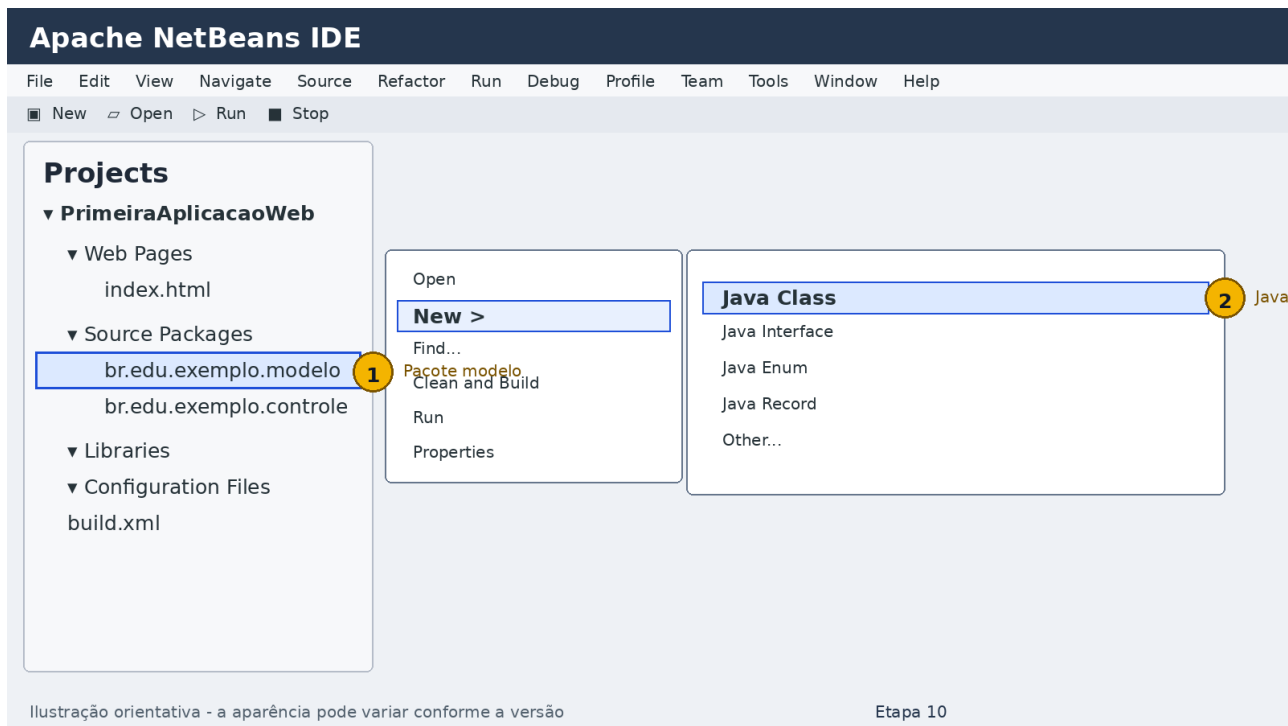


Figura 10 - Criando a classe Java dentro do pacote modelo.

Dê à classe o nome Pessoa e substitua o conteúdo pelo código a seguir.

```
package br.edu.exemplo.modelo;

public class Pessoa {
    private String nome;
    private int idade;

    public Pessoa() { }

    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public int getIdade() { return idade; }
    public void setIdade(int idade) { this.idade = idade; }

    public String getFaixaEtaria() {
        if (idade < 12) return "Criança";
        if (idade < 18) return "Adolescente";
        if (idade < 60) return "Adulto";
        return "Idoso";
    }
}
```

A classe aplica encapsulamento, construtores e uma regra de negócio. A novidade é que o objeto será criado durante uma requisição HTTP.

## 8. Criar o formulário HTML

Clique com o botão direito em Web Pages e selecione New → HTML File. Use o nome index.

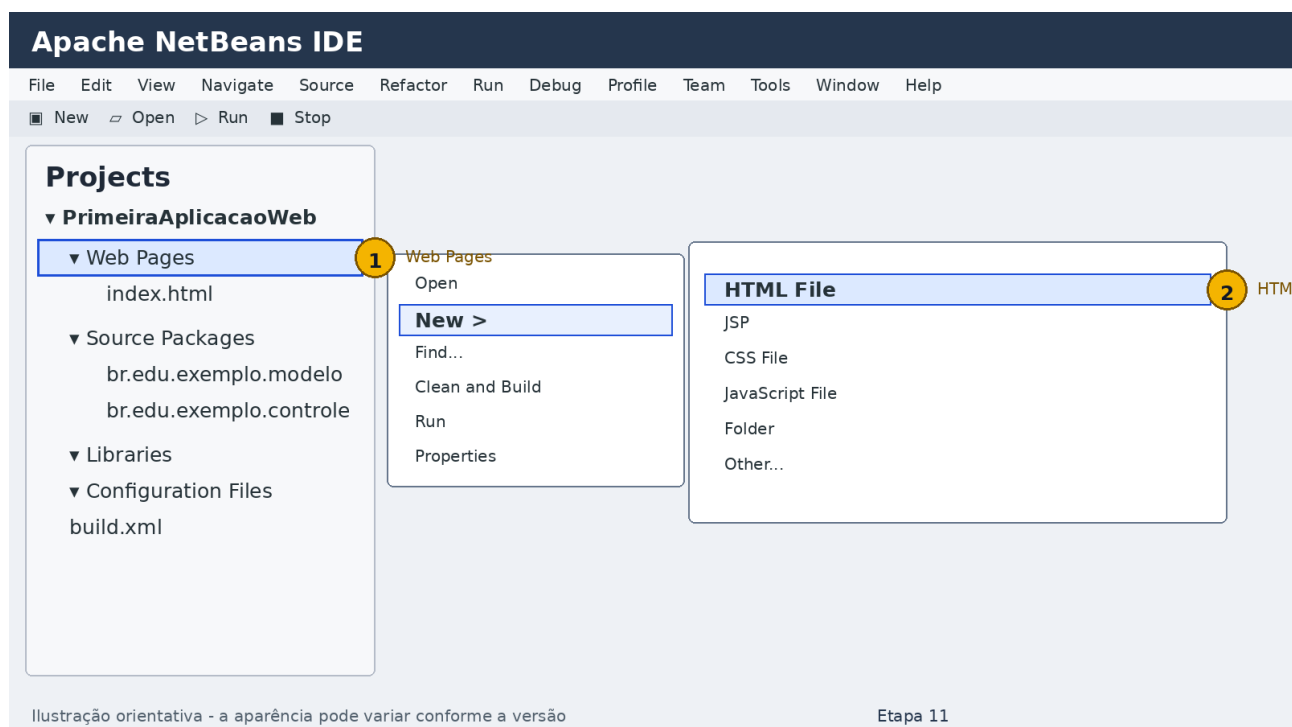


Figura 11 - Criando o arquivo index.html.

## Código do formulário

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cadastro de pessoa</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f6f8;
          color: #222; margin: 0; padding: 40px; }
    main { max-width: 500px; margin: auto; background: white;
          padding: 30px; border: 1px solid #ccc; border-radius: 8px; }
    label { display: block; margin-top: 16px; font-weight: bold; }
    input { box-sizing: border-box; width: 100%; margin-top: 6px;
          padding: 10px; border: 1px solid #555; border-radius: 4px; }
    button { width: 100%; margin-top: 24px; padding: 12px; border: 0;
            border-radius: 4px; background: #1d4ed8; color: white;
            font-size: 16px; cursor: pointer; }
    button:hover { background: #163c7d; }
  </style>
</head>
<body>
<main>
  <h1>Cadastro de pessoa</h1>
  <p>Preencha os campos para enviar os dados ao servidor.</p>
  <form action="cadastro" method="post">
    <label for="nome">Nome</label>
    <input type="text" id="nome" name="nome" required>
    <label for="idade">Idade</label>
    <input type="number" id="idade" name="idade"
          min="0" max="130" required>
    <button type="submit">Cadastrar</button>
  </form>
</main>
</body>
</html>
```



## Entendendo o formulário

| Trecho            | Função                                   |
|-------------------|------------------------------------------|
| action="cadastro" | Endereço que receberá os dados.          |
| method="post"     | Envia os dados usando HTTP POST.         |
| name="nome"       | Nome do parâmetro recuperado no Servlet. |
| required          | Validação inicial feita pelo navegador.  |
| type="number"     | Campo numérico com limites definidos.    |

Os atributos name precisam corresponder exatamente aos nomes passados para `request.getParameter()`.

## 9. Criar o Servlet

Clique com o botão direito em Source Packages e selecione New → Servlet. Se a opção não estiver visível, use New → Other → Web → Servlet.

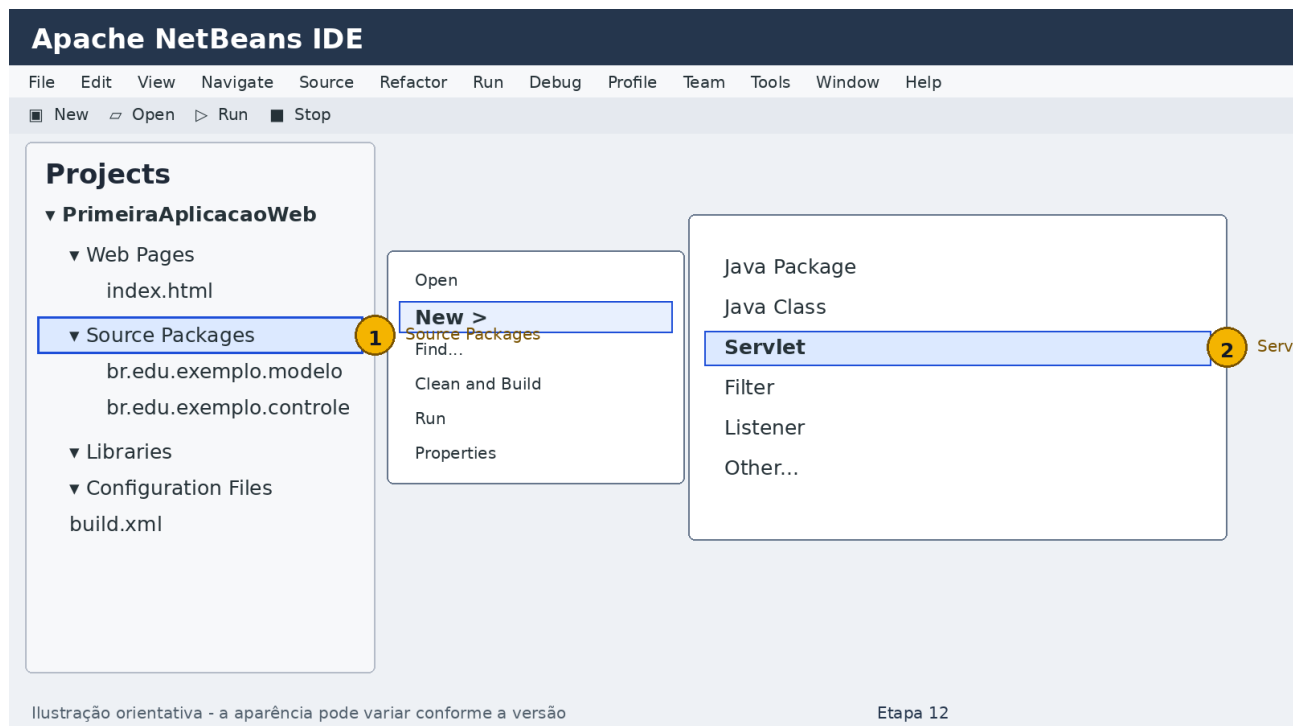


Figura 12 - Abrindo o assistente de criação de Servlet.



Figura 13 - Definindo classe, pacote e padrão de URL.

Use a classe `CadastroServlet`, o pacote `br.edu.exemplo.controle` e confirme o endereço `/cadastro`.

## Código inicial do Servlet

```
package br.edu.exemplo.controle;

import br.edu.exemplo.modelo.Pessoa;
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/cadastro")
public class CadastroServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");

        String nome = request.getParameter("nome");
        String idadeTexto = request.getParameter("idade");
        int idade = Integer.parseInt(idadeTexto);

        Pessoa pessoa = new Pessoa(nome, idade);
        request.setAttribute("pessoa", pessoa);

        request.getRequestDispatcher("/resultado.jsp")
            .forward(request, response);
    }
}
```

## Como o Servlet funciona

| Código                         | Significado                                     |
|--------------------------------|-------------------------------------------------|
| @WebServlet("/cadastro")       | Associa a URL /cadastro à classe.               |
| extends HttpServlet            | Especializa a classe para trabalhar com HTTP.   |
| doPost(...)                    | Atende requisições POST.                        |
| getParameter("nome")           | Lê um parâmetro enviado pelo formulário.        |
| setAttribute("pessoa", pessoa) | Disponibiliza o objeto para a JSP.              |
| forward(...)                   | Encaminha a mesma requisição para outra página. |

**Parâmetro: dado enviado pelo cliente. Atributo: dado colocado no servidor durante o processamento.**

## 10. Criar a página JSP

Clique com o botão direito em Web Pages e selecione New → JSP. Informe o nome resultado.

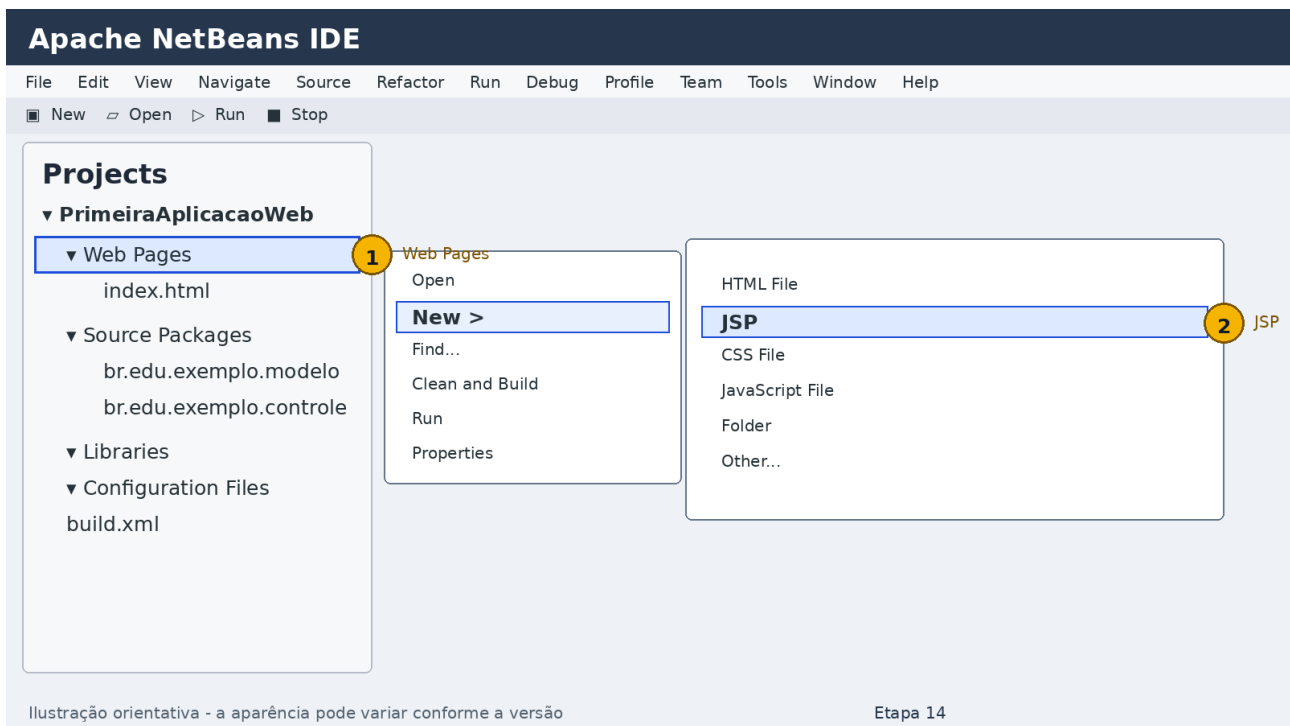


Figura 14 - Criando resultado.jsp.

## Código da página resultado.jsp

```
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Resultado do cadastro</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f6f8;
      color: #222; margin: 0; padding: 40px; }
    main { max-width: 500px; margin: auto; background: white;
      padding: 30px; border: 1px solid #ccc; border-radius: 8px; }
    .resultado { padding: 16px; background: #e8f0fe;
      border-left: 5px solid #1d4ed8; }
    a { display: inline-block; margin-top: 20px; color: #1644aa; }
  </style>
</head>
<body>
<main>
  <h1>Cadastro realizado</h1>
  <div class="resultado">
    <p><strong>Nome:</strong> ${pessoa.nome}</p>
    <p><strong>Idade:</strong> ${pessoa.idade}</p>
    <p><strong>Faixa etária:</strong> ${pessoa.faixaEtaria}</p>
  </div>
  <a href="index.html">Realizar outro cadastro</a>
</main>
</body>
</html>
```

As expressões `${pessoa.nome}`, `${pessoa.idade}` e `${pessoa.faixaEtaria}` usam Expression Language. Elas acessam os métodos getters do objeto Pessoa.

# 11. Executar e testar

Salve todos os arquivos. Selecione o projeto e use Run → Run Project, pressione F6 ou clique no botão verde de execução.



Figura 15 - Executando o projeto no Tomcat.





Figura 16 - Aplicação aberta no navegador.

Preencha nome e idade e clique em Cadastrar. O formulário envia uma requisição POST para o Servlet, que cria o objeto e encaminha para a JSP.

## Endereço esperado

```
http://localhost:8080/PrimeiraAplicacaoWeb/
```

## Fluxo completo

**index.html → POST /cadastro → CadastroServlet → objeto Pessoa → resultado.jsp → HTML no navegador**

## 12. Adicionar validação no servidor

A validação HTML melhora a experiência, mas pode ser contornada. O Servlet também deve validar os parâmetros.

```
@Override
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    request.setCharacterEncoding("UTF-8");
    String nome = request.getParameter("nome");
    String idadeTexto = request.getParameter("idade");

    if (nome == null || nome.isBlank()) {
        encaminharErro(request, response, "O nome deve ser informado.");
        return;
    }

    try {
        int idade = Integer.parseInt(idadeTexto);
        if (idade < 0 || idade > 130) {
            encaminharErro(request, response,
                           "A idade deve estar entre 0 e 130 anos.");
            return;
        }

        Pessoa pessoa = new Pessoa(nome, idade);
        request.setAttribute("pessoa", pessoa);
        request.getRequestDispatcher("/resultado.jsp")
            .forward(request, response);

    } catch (NumberFormatException e) {
        encaminharErro(request, response, "A idade informada não é válida.");
    }
}

private void encaminharErro(HttpServletRequest request,
                          HttpServletResponse response,
                          String mensagem)
    throws ServletException, IOException {
    request.setAttribute("erro", mensagem);
    request.getRequestDispatcher("/erro.jsp")
        .forward(request, response);
}
```

## Crie erro.jsp

```
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Erro no cadastro</title>
</head>
<body>
  <main>
    <h1>Não foi possível realizar o cadastro</h1>
    <p>${erro}</p>
    <a href="index.html">Voltar ao formulário</a>
  </main>
</body>
</html>
```

Teste valores válidos, nome vazio, idade negativa, idade superior a 130 e texto no parâmetro de idade.

## 13. Erros comuns e soluções

| Sintoma                               | Causa provável e solução                                                                             |
|---------------------------------------|------------------------------------------------------------------------------------------------------|
| Pacote jakarta.servlet não encontrado | Confira se o projeto está associado ao Tomcat e se as bibliotecas do servidor aparecem em Libraries. |
| Erro 404 em /cadastro                 | Confira @WebServlet("/cadastro"), o action do formulário e se o projeto foi implantado.              |
| Erro 500 ao enviar                    | Leia o log do Tomcat. Verifique conversão da idade e nomes dos parâmetros.                           |
| Acentos incorretos                    | Mantenha UTF-8 no HTML, na JSP e request.setCharacterEncoding("UTF-8").                              |
| Servidor não inicia                   | Uma porta pode estar ocupada. Pare outra instância do Tomcat ou altere a porta.                      |
| Código usa javax.servlet              | Troque os imports por jakarta.servlet ao usar Tomcat 10 ou 11.                                       |
| JSP não recebe o objeto               | Confira setAttribute("pessoa", pessoa) e \${pessoa...}.                                              |
| Página inicial não abre               | Confirme que index.html está diretamente em Web Pages.                                               |

Para acompanhar o servidor, abra Window → Output e selecione a saída do Apache Tomcat.

## 14. Exercício prático

Amplie a aplicação para cadastrar um aluno. Substitua Pessoa por uma classe Aluno com os seguintes atributos:

- nome;
- e-mail;
- curso;
- semestre;
- turno.

### Requisitos

- Criar um formulário de cadastro de aluno.
- Exigir nome e e-mail.
- Permitir selecionar o curso em uma lista.
- Receber o semestre como número.
- Permitir selecionar o turno.
- Criar um objeto Aluno no Servlet.
- Encaminhar o objeto para uma JSP.
- Apresentar todos os dados cadastrados.
- Exibir uma página de erro quando houver dados inválidos.

### Esqueleto da classe

```
package br.edu.exemplo.modelo;

public class Aluno {
    private String nome;
    private String email;
    private String curso;
    private int semestre;
    private String turno;

    public Aluno() { }

    public Aluno(String nome, String email, String curso,
        int semestre, String turno) {
        this.nome = nome;
        this.email = email;
        this.curso = curso;
        this.semestre = semestre;
        this.turno = turno;
    }

    // Gere getters e setters pelo NetBeans.
}
```

## 15. Síntese da aula

A primeira aplicação estabeleceu a ponte entre a orientação a objetos e o desenvolvimento web.

| Conhecimento anterior          | Aplicação na web                                                  |
|--------------------------------|-------------------------------------------------------------------|
| Classe e objeto                | Pessoa é criada durante a requisição.                             |
| Encapsulamento                 | A JSP acessa getters por Expression Language.                     |
| Métodos                        | doPost processa o formulário; getFaixaEtaria aplica uma regra.    |
| Exceções                       | NumberFormatException trata idade inválida.                       |
| Separação de responsabilidades | HTML coleta, Servlet controla, modelo representa e JSP apresenta. |

### Sequência principal

**Formulário → requisição HTTP → Servlet → objeto Java → atributo de requisição → JSP → resposta HTML**

### Próxima aula sugerida

Aprofundar GET e POST, ciclo de vida dos Servlets, redirecionamento versus encaminhamento e desenvolver um CRUD provisório em memória, ainda sem banco de dados.

### Checklist final

- Tomcat configurado e iniciado.
- Projeto Web criado com Ant.
- Imports usando jakarta.servlet.
- Formulário envia POST para cadastro.
- Servlet cria Pessoa e usa forward.
- JSP apresenta os getters com Expression Language.
- Validação testada no navegador e no servidor.